

验重方法笔记

验重常用查询语句

使用COUNT(1)

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 检查是否存在 -->
    <select id="countByUsername" resultType="int">
        SELECT COUNT(1) FROM users
        WHERE username = #{username}
        <if test="excludeId != null and excludeId != ''">
            AND id != #{excludeId} <!-- 编辑时排除自身 -->
        </if>
    </select>
</mapper>
```

```
public boolean existsByUsername(String username, String excludeId) {
    int count = userMapper.countByUsername(username, excludeId);
    return count > 0;
}
```

EXISTS子查询

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 检查是否存在 -->
    <select id="existsByUsername" resultType="boolean">
        SELECT CASE WHEN EXISTS (
            SELECT 1 FROM users
            WHERE username = #{username}
            <if test="excludeId != null and excludeId != ''">
                AND id != #{excludeId} <!-- 编辑时排除自身 -->
            </if>
        ) THEN 1 ELSE 0 END AS exists_flag
    </select>
</mapper>
```

```
public boolean existsByUsername(String username, String excludeId) {
    return userMapper.existsByUsername(username, excludeId);
}
```

使用LIMIT子句

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 检查是否存在 -->
    <select id="existsByUsernameWithLimit" resultType="boolean">
        SELECT 1 FROM users
        WHERE username = #{username}
        <if test="excludeId != null and excludeId != ''">
            AND id != #{excludeId} <!-- 编辑时排除自身 -->
        </if>
        LIMIT 1
    </select>
</mapper>
```

```
public boolean existsByUsernameWithLimit(String username, String excludeId) {
    return userMapper.existsByUsernameWithLimit(username, excludeId);
}
```

以上三种：

- 优点：简单易懂，适合只需判断存在与否的场景
- 缺点：高并发下存在幻读问题

使用IN子查询

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 批量检查是否存在 -->
    <select id="existsByUsernames" resultType="string">
        SELECT username FROM users
        WHERE username IN
        <foreach item="username" collection="usernames" open="(" separator=","
close=")">
            #{username}
        </foreach>
    </select>
</mapper>
```

```
public Set<String> existsByUsernames(List<String> usernames) {
    List<String> existingUsernames = userMapper.existsByUsernames(usernames);
    return new HashSet<>(existingUsernames);
}
```

- 优点：适合批量验重，减少数据库查询次数
- 缺点：IN列表过长时性能下降

使用MERGE语句

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 使用MERGE进行插入或更新 -->
    <insert id="mergeUser">
        MERGE INTO users AS target
        USING (SELECT #{username} AS username, #{email} AS email) AS source
        ON target.username = source.username
        WHEN MATCHED THEN
            UPDATE SET email = source.email
        WHEN NOT MATCHED THEN
            INSERT (username, email) VALUES (source.username, source.email);
    </insert>
</mapper>
```

```
public void mergeUser(User user) {
    userMapper.mergeUser(user);
}
```

- 优点：适合插入或更新操作，减少代码复杂度，不用担心并发问题
- 缺点：数据库支持有限，语法差异较大，不能完全替代验重逻辑

ON DUPLICATE KEY UPDATE （MySQL 特有语法）

利用 MySQL 的原子性操作，将“查重”和“插入”合并为一步，避免应用层的 race condition。

```
<!-- UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <!-- 重复则更新 (Upsert) -->
    <insert id="upsertUser">
        INSERT INTO users (username, email)
        VALUES (#{username}, #{email})
        ON DUPLICATE KEY UPDATE
            email = VALUES(email)
    </insert>
</mapper>
```

```
public boolean registerUser(User user) {
    userMapper.upsertUser(user);
    return true;
}
```

- 优点：数据库原子指令，线程安全，无额外网络开销。
- 缺点：仅适用于 MySQL，且只能处理插入或更新操作，无法单纯用于验重。

业务逻辑验重

自定义注解校验

在 Spring Boot 开发中，自定义注解可以将繁琐的业务校验逻辑从业务代码中剥离，实现声明式校验。

工作原理与流程：

1. 定义注解：创建一个自定义注解（如 `@UniqueData`），指定其元数据和校验器类。
2. 实现校验器：实现校验器接口，注入业务服务，编写具体的校验逻辑。
3. 声明式使用：在 DTO（数据传输对象）的字段上直接标记该注解。
4. 框架触发：在 Controller 层接收参数时，使用 `@Valid` 或 `@Validated` 注解触发校验机制。Spring 框架会在数据绑定阶段调用校验器，如果校验失败，自动抛出异常或绑定错误信息。

适用性分析：

- 优势：极大提升了代码的可读性和整洁度，避免了在 Service 层充斥着大量校验逻辑。
- 局限：
 - 并发控制：校验通常发生在 Controller 参数绑定阶段，此时尚未进入业务方法，难以像 Service 层那样方便地控制事务或加分布式锁，高并发下容易出现“检查通过但插入失败”的情况。
 - 复杂逻辑：对于依赖多字段联合校验或复杂业务上下文的场景，注解校验可能能力不足。
- 优点：代码简洁，逻辑解耦，符合规范。
- 缺点：仅适用于非高并发、对即时一致性要求不高的场景。

使用MyBatis-Plus

```
public boolean existsByUsername(String username, String excludeId) {  
    return userMapper.exists(new LambdaQueryWrapper<User>()  
        .eq(User::getUsername, username)  
        .ne(StringUtils.isNotBlank(excludeId), User::getId, excludeId));  
}
```

MP 不需要手写 SQL，且直接提供了 `.exists()` 方法支持，底层自动优化为 `SELECT 1、LIMIT 1` 等。

大数据量下的验重

布隆过滤器

布隆过滤器是一种空间效率很高的概率型数据结构，用于判断一个元素是否在一个集合中。

工作原理：

1. 初始化：创建一个包含 m 位的位数组（Bit Array），并将所有位初始化为 0。
2. 映射：定义 k 个不同的哈希函数，每个函数将输入元素映射到位数组中的一个位置。

3. 添加元素：将要添加的元素分别通过k个哈希函数计算，得到k个位置，将位数组中对应的位置都置为1。
4. 查询元素：将要查询的元素分别通过k个哈希函数计算，得到k个位置。
 - 如果所有位置的位都为1，则该元素可能存在。
 - 如果有任意一个位置的位为0，则该元素一定不存在。
5. 误判：由于哈希冲突的存在，不同的元素可能映射到相同的位置，导致查询时误判为存在，但绝不会误判为不存在。

应用流程：

1. 快速筛选：在执行昂贵的数据库查询前，先通过布隆过滤器查一遍。
 2. 过滤不存在：如果布隆过滤器判定不存在，则直接返回校验通过。
 3. 二次确认：如果布隆过滤器判定可能存在，则继续查询数据库进行确认，以排除误判情况。
- 优点：适合大数据量场景，内存占用极低，查询速度极快 ($O(k)$)。
 - 缺点：存在误判率，不支持删除操作（标准布隆过滤器）。

bitmap位图

Bitmap（位图）是一种使用二进制位来标记元素是否存在的数据结构。

工作原理：

1. 映射：将集合中的元素映射到一个整数空间（如ID、哈希值），通常该空间是连续的。
2. 存储：申请一个足够大的位数组，用第N位的值来表示整数N是否存在。
 - 0：表示不存在。
 - 1：表示存在。
3. 操作：通过位运算可以极其高效地进行查询、添加和集合运算。

应用场景与限制：

- 适用：适用于海量数据的快速查找、去重、排序，特别是当数据是密集的整数ID时。
- 不适用：由于位图的大小取决于数据的最大值而非数量，如果数据稀疏会造成极大的空间浪费。
- 非整数处理：对于字符串等非整数数据，通常需要配合哈希算法将其转换为整数，但这引入了哈希冲突的风险，类似于布隆过滤器，可能需要二次验证。
- 优点：内存占用极小，查询和修改速度为 $O(1)$ 。
- 缺点：数据稀疏时空间利用率低，原生仅支持整数，处理冲突复杂。

数据库约束验重

唯一索引

```
ALTER TABLE `student_info` ADD UNIQUE INDEX `idx_student_no` (`student_no`);
```

- 优点：简单高效，数据库层面保证数据唯一性，性能更好，没有并发问题，可移植性强
- 缺点：错误信息不友好，无法自定义逻辑

触发器

```
CREATE TRIGGER before_student_insert
BEFORE INSERT ON student_info
FOR EACH ROW
BEGIN
    DECLARE duplicate_count INT;

    SELECT COUNT(*) INTO duplicate_count
    FROM student_info
    WHERE student_no = NEW.student_no
    FOR UPDATE; -- 加锁防止并发问题

    IF duplicate_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Duplicate student number';
    END IF;
END;
```

- 优点：可以在插入前执行复杂逻辑，可以自定义错误消息
- 缺点：并发问题（需要加锁解决），维护复杂

SELECT ... FOR UPDATE (悲观锁)

```
SELECT 1 FROM users WHERE username = #{username} FOR UPDATE
```

- 场景：严格要求一致性，且并发量不大的内部系统。
- 局限：会阻塞其他事务。

其他

使用Redis缓存提升性能

```
public boolean checkUsernameDuplicate(String username) {
    String key = USERNAME_KEY + username;

    // 先从Redis检查
    Boolean exists = redisTemplate.hasKey(key);
    if (Boolean.TRUE.equals(exists)) return true;

    // Redis中没有，查数据库
    boolean dbExists = userMapper.existsByUsername(username) > 0;

    // 如果存在，缓存到Redis
    if (dbExists) {
        redisTemplate.opsForValue().set(key, "1", EXPIRE_TIME, TimeUnit.SECONDS);
    }

    return dbExists;
}
```

使用索引优化查询性能

```
CREATE INDEX idx_username ON users(username);
```

分布式锁 (Redisson)

在高并发场景下，先查询后插入无法完全避免重复数据，需要加锁将操作串行化。尤其是双重检查锁，确保在锁内再次验证数据唯一性。

- 优点：彻底解决应用层并发导致的重复插入问题（幻读）。
- 缺点：性能有所损耗，设计不当会降低吞吐量。

逻辑删除对验重的影响

现在的系统（包括 JeecgBoot）大多采用逻辑删除（del_flag），这会给验重和唯一索引带来巨大的坑。

- 业务逻辑问题：如果一个用户注销了（del_flag=1），可以注册一个同名的新用户吗？
 - 若可以：你的 SQL 必须加上 AND del_flag = 0。
 - 若不可以：验重逻辑需要查全量数据。
- 唯一索引冲突：如果数据库加了唯一索引 UNIQUE(username)，当一个用户被逻辑删除后，数据库里依然有这条记录。此时插入同名新用户会报 Duplicate entry 错误。
 - 解决方案 A：唯一索引带上删除字段 UNIQUE(username, del_flag)。
 - 缺陷：通常删除后的 del_flag 都是 1，导致只能删除一次，第二次删除同名用户时会冲突。
 - 解决方案 B：删除时将 del_flag 更新为 id 或时间戳，或者删除时修改 username (如 username_deleted_timestamp)。

反面典型

COUNT(*)

```
<select id="countByUsername" resultType="int">
    SELECT COUNT(*) FROM users WHERE username = #{username}
</select>
```

- COUNT(*) 会扫描符合条件的所有记录，性能较差，且无法避免并发插入导致的幻读问题。